Promise Combinators

Pick by what you need: all, allSettled, race or any

Method	Resolves with	Rejects when	Empty []
`Promise.all`	all values	any one rejects	fulfills []
`Promise.allSettled`	all outcomes	never	fulfills []
`Promise.race`	first settled	first settled rejects	pending forever
`Promise.any`	first fulfilled	all reject	rejects

1 race: add a timeout

```
Promise.race([
  fetchData(),   // may be slow
  timeout(3000), // your helper
]);
// race never cancels the loser
// use AbortController for that
```

2 allSettled never rejects

```
const ok = Promise.resolve("A");
const no = Promise.reject("B");
const r = await Promise
  .allSettled([ok, no]);
r[0].status;   // "fulfilled"
r[1].status;   // "rejected"
```

TRAP What does this log?

```
Promise.any([
  Promise.reject(1),
  Promise.reject(2),
]).catch(e => console.log(e.name));
```

`AggregateError`

If every promise rejects, **any** rejects with an **AggregateError**. The reasons are in **e.errors**: **[1, 2]**.

KEY TAKEAWAY

- `all` — You need every result
- `allSettled` — You need every outcome
- `race/any` — First settled / first success

? Follow-up questions to expect

- `all` vs `allSettled`? → all fails on the first rejection, allSettled never fails.
- `race` vs `any`? → race: first settled. any: first fulfilled.
- **Result order? Plain values?** → Input order. Plain values count as fulfilled.
- **How do you add a timeout to a request?** → race it with a timer; abort with AbortController.